

//Implement binary tree

```
#include <iostream>
```

```
using namespace std;
```

```
class Node {
```

```
public:
```

```
    int data;
```

```
    Node* left;
```

```
    Node* right;
```

```
    Node(int value) : data(value), left(nullptr), right(nullptr) {}  
};
```

```
class BinaryTree {
```

```
public:
```

```
    Node* root;
```

```
    BinaryTree() : root(nullptr) {}
```

```
    void insertRoot(int value) {
```

```
        if (root == nullptr) {
```

```
            root = new Node(value);
```

```
        } else {
```

```
            cout << "Root already exists!" << endl;
```

```
        }
```

```
    }
```

```
    void insertLeft(Node* parent, int value) {
```

```
        if (parent->left == nullptr) {
```

```
            parent->left = new Node(value);
```

```
        } else {
```

```
            cout << "Left child already exists!" << endl;
```

```
        }
```

```
    }
```

```
    void insertRight(Node* parent, int value) {
```

```
        if (parent->right == nullptr) {
```

```
            parent->right = new Node(value);
```

```
        } else {
```

```
            cout << "Right child already exists!" << endl;
```

```
        }
```

```
    }
```

```
};
```

```
int main() {
```

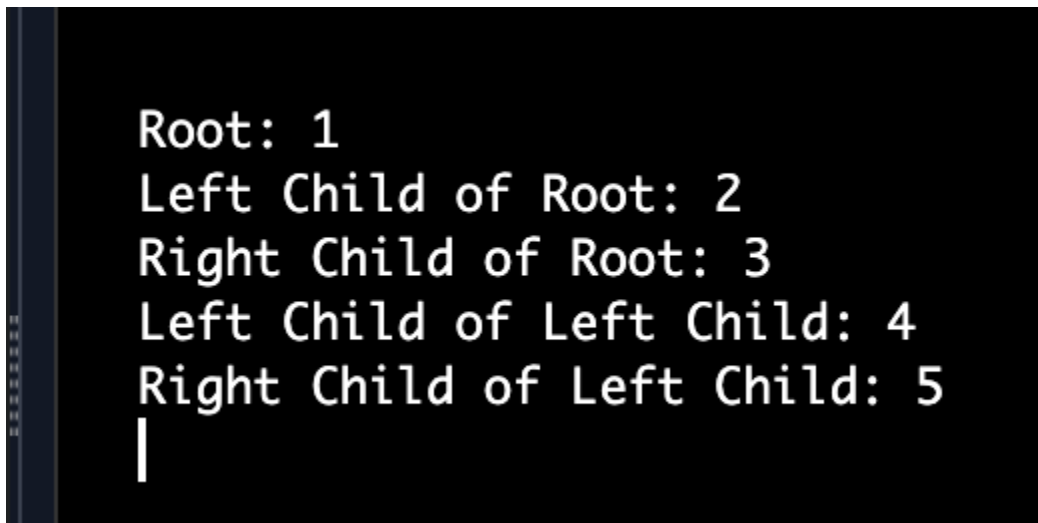
```

BinaryTree tree;
tree.insertRoot(1);
tree.insertLeft(tree.root, 2);
tree.insertRight(tree.root, 3);
tree.insertLeft(tree.root->left, 4);
tree.insertRight(tree.root->left, 5);

cout << "Root: " << tree.root->data << endl;
cout << "Left Child of Root: " << tree.root->left->data << endl;
cout << "Right Child of Root: " << tree.root->right->data << endl;
cout << "Left Child of Left Child: " << tree.root->left->left->data << endl;
cout << "Right Child of Left Child: " << tree.root->left->right->data << endl;

return 0;
}

```



```

Root: 1
Left Child of Root: 2
Right Child of Root: 3
Left Child of Left Child: 4
Right Child of Left Child: 5
|

```

//Tree traversal

```

#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* left;
    Node* right;

    Node(int value) : data(value), left(nullptr), right(nullptr) {}
};

class BinaryTree {

```

```

public:
    Node* root;
    BinaryTree() : root(nullptr) {}

    void insertRoot(int value) {
        if (root == nullptr) {
            root = new Node(value);
        } else {
            cout << "Root already exists!" << endl;
        }
    }

    void insertLeft(Node* parent, int value) {
        if (parent->left == nullptr) {
            parent->left = new Node(value);
        } else {
            cout << "Left child already exists!" << endl;
        }
    }

    void insertRight(Node* parent, int value) {
        if (parent->right == nullptr) {
            parent->right = new Node(value);
        } else {
            cout << "Right child already exists!" << endl;
        }
    }

    void inOrder(Node* node) {
        if (node == nullptr) return;
        inOrder(node->left);
        cout << node->data << " ";
        inOrder(node->right);
    }

    void preOrder(Node* node) {
        if (node == nullptr) return;
        cout << node->data << " ";
        preOrder(node->left);
        preOrder(node->right);
    }

    void postOrder(Node* node) {
        if (node == nullptr) return;
    }

```

```

        postOrder(node->left);
        postOrder(node->right);
        cout << node->data << " ";
    }
};

int main() {
    BinaryTree tree;
    tree.insertRoot(1);
    tree.insertLeft(tree.root, 2);
    tree.insertRight(tree.root, 3);
    tree.insertLeft(tree.root->left, 4);
    tree.insertRight(tree.root->left, 5);

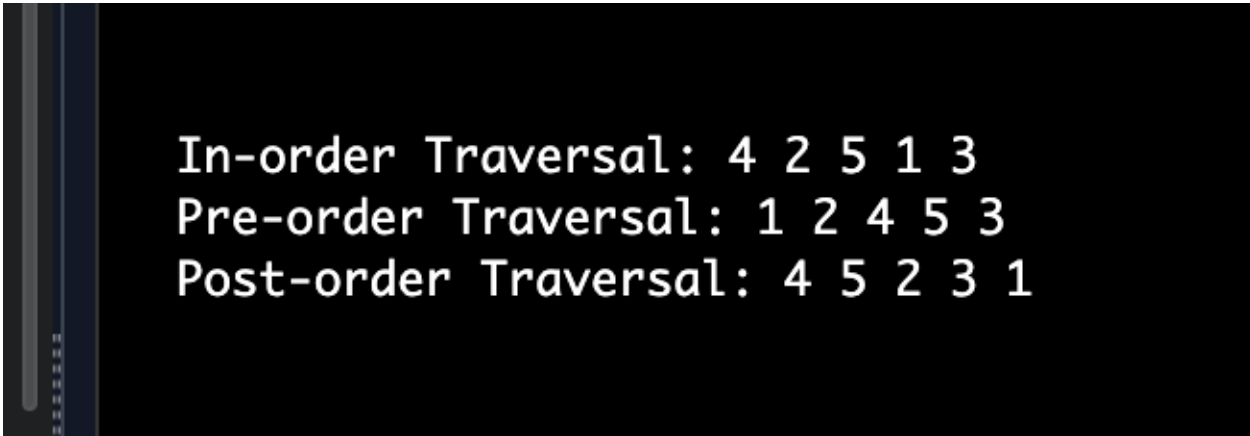
    cout << "In-order Traversal: ";
    tree.inOrder(tree.root);
    cout << endl;

    cout << "Pre-order Traversal: ";
    tree.preOrder(tree.root);
    cout << endl;

    cout << "Post-order Traversal: ";
    tree.postOrder(tree.root);
    cout << endl;

    return 0;
}

```



```

In-order Traversal: 4 2 5 1 3
Pre-order Traversal: 1 2 4 5 3
Post-order Traversal: 4 5 2 3 1

```

```

//Binary Search Tree
#include <iostream>
using namespace std;

```

```

class Node {
public:
    int data;
    Node* left;
    Node* right;

    Node(int value) : data(value), left(nullptr), right(nullptr) {}
};

```

```

class BinaryTree {
public:
    Node* root;
    BinaryTree() : root(nullptr) {}

    Node* insert(Node* node, int key) {
        if (node == nullptr) {
            return new Node(key);
        }
        if (key < node->data) {
            node->left = insert(node->left, key);
        } else if (key > node->data) {
            node->right = insert(node->right, key);
        }
        return node;
    }
}

```

```

Node* search(Node* node, int key) {
    if (node == nullptr || node->data == key) {
        return node;
    }
    if (key < node->data) {
        return search(node->left, key);
    }
    return search(node->right, key);
}

```

```

Node* deleteNode(Node* root, int key) {
    if (root == nullptr) return root;
    if (key < root->data) {
        root->left = deleteNode(root->left, key);
    } else if (key > root->data) {
        root->right = deleteNode(root->right, key);
    } else {
        if (root->left == nullptr) {

```

```

        Node* temp = root->right;
        delete root;
        return temp;
    } else if (root->right == nullptr) {
        Node* temp = root->left;
        delete root;
        return temp;
    }
    Node* temp = minValueNode(root->right);
    root->data = temp->data;
    root->right = deleteNode(root->right, temp->data);
}
return root;
}

Node* minValueNode(Node* node) {
    Node* current = node;
    while (current && current->left != nullptr) {
        current = current->left;
    }
    return current;
}

void inOrder(Node* node) {
    if (node == nullptr) return;
    inOrder(node->left);
    cout << node->data << " ";
    inOrder(node->right);
}
};

```

```

int main() {
    BinaryTree bst;
    bst.root = bst.insert(bst.root, 8);
    bst.insert(bst.root, 3);
    bst.insert(bst.root, 10);
    bst.insert(bst.root, 1);
    bst.insert(bst.root, 6);
    bst.insert(bst.root, 4);
    bst.insert(bst.root, 7);
    bst.insert(bst.root, 14);
    bst.insert(bst.root, 13);

    cout << "In-order traversal before deletion: ";
}

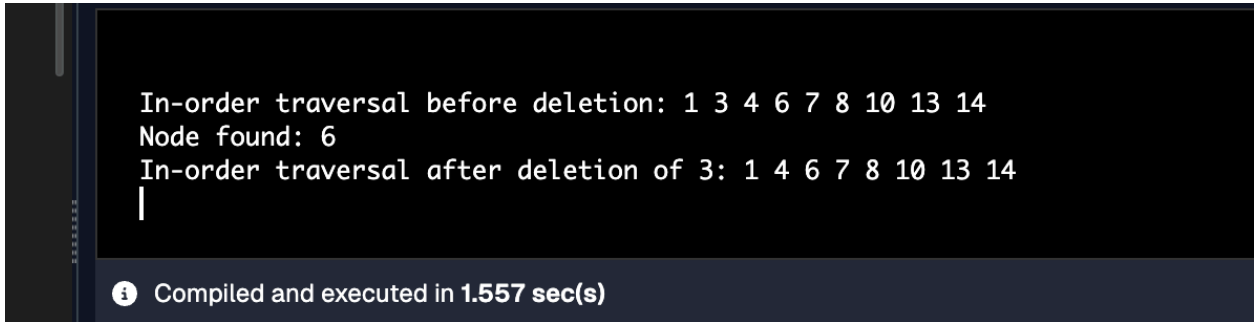
```

```
bst.inOrder(bst.root);
cout << endl;

Node* result = bst.search(bst.root, 6);
if (result != nullptr) {
    cout << "Node found: " << result->data << endl;
} else {
    cout << "Node not found" << endl;
}

bst.root = bst.deleteNode(bst.root, 3);
cout << "In-order traversal after deletion of 3: ";
bst.inOrder(bst.root);
cout << endl;

return 0;
}
```



```
In-order traversal before deletion: 1 3 4 6 7 8 10 13 14
Node found: 6
In-order traversal after deletion of 3: 1 4 6 7 8 10 13 14
|
```

 Compiled and executed in 1.557 sec(s)